Sensor Identity Does Not Transfer: Revisiting Test-Time Adaptation for Cross-Building Human Activity Recognition

Mariem Kbaier[a], Ons Abderrahim[a], Manar Amayri[a] and Nizar Bouguila[a,*]

[a]*Concordia Institute for Information Systems Engineering, Concordia University, Montreal, QC, Canada*

ARTICLE INFO

Keywords:
Test-time adaptation
Human activity recognition
Occupancy estimation
Smart buildings
Evaluation leakage

ABSTRACT

Test-time adaptation (TTA) is an appealing answer to cross-building deployment in smart buildings: sensors run continuously, and labelled data from a new site never arrives. We evaluate three TTA methods developed for images (DIGA, PETAL and PeTTA) on human activity recognition and occupancy estimation across buildings, and find that how much they help is governed by the quality of the model they start from and not by the method. Occupancy estimation supplies the cleanest test, because the two transfer directions differ only in the quality of the source checkpoint: against a sound one no method improves and the normalisation-based ones lose ground, while against a checkpoint worse than chance the same methods gain 27 to 41 MCC points. Adaptation gain here measures source-model deficiency, not adaptation quality.

Reaching that comparison required measuring the cross-building gap itself. Sliding windows with unit stride cause adjacent samples to share nineteen of twenty timesteps, so a random train/validation split places a near-duplicate of almost every validation sample into training; under that split a source model reaches 0.998 macro-F_1, and grouped by recording day it reaches 0.53. The window definition makes almost no difference to this: the inflation comes from the split. Features matter as much. The 106 per-sensor features that dominate the usual design score *below* stratified-random guessing on the target in both directions, because a given sensor index is a different physical device in each building. With causal windows that an online system can compute, and thirteen window-level summary statistics that name no sensor, a model transfers to the second apartment within 0.6 macro-F_1 points of a model trained on that apartment's own days, at matched training-set size and on the same held-out days. Entering the first apartment the same comparison costs 21.3 points. One of the two directions therefore contains essentially no domain shift, which the conventional source-minus-target figure of 8.8 points conceals by charging the transfer for a difference between the buildings.

Against that baseline all four methods act through one channel: replacing source batch-normalisation statistics with the test stream's, which each released implementation makes its primary mechanism. Disabling that channel and leaving everything else intact, PETAL and PeTTA alter fewer than 25 of 3412 predictions, so their gradient updates, posterior regularisation and divergence gating contribute almost nothing. Enabling it as their own code configures it, PETAL lands within 0.3 macro-F_1 points of plain AdaBN and PeTTA within 1.1. The cost of the channel rises monotonically with the share of target statistics mixed in, and the only component that helps anywhere without target labels is DIGA's prototype classifier, worth 4.4 points in the direction with headroom and −5.7 in the direction without. Across two tasks, two directions and four methods, every method gains 28 to 43 MCC points against a checkpoint worse than chance and loses against every sound one. Our conclusion is not that TTA fails in smart buildings, but that it was being asked to repair a problem largely created by evaluation and feature design.

1. Introduction

Buildings account for a large share of global energy demand, and occupancy-aware control of HVAC and lighting is one of the more direct levers available for reducing it [2, 3]. Human activity recognition (HAR) and occupancy estimation (OE) supply the signal such control needs [4, 5]. Both are learned from sensor traces, and both face the same obstacle when a model trained in one building is installed in another: room layouts differ, sensors sit in different places, and occupants keep different routines [7, 8]. A model that works in the building where it was trained often does not work in the next one.

Test-time adaptation addresses this without requiring labels from the new site. A frozen source model updates itself online from the unlabelled stream it is already processing, with no access to source data and no target annotation [9]. For buildings, where sensors are always on and manual annotation is impractical, the fit is obvious. The method family has matured quickly on images: DIGA [10] adapts normalisation statistics and class prototypes without gradients, PETAL [11] treats lifelong adaptation as probabilistic inference, and PeTTA [12] stabilises adaptation when environments recur. Whether their assumptions survive the move to one-dimensional sensor data is an open question, and the one we set out to answer.

Answering it requires first establishing that the quantity these methods are asked to reduce is measured soundly, and we began there. Implementing the standard pipeline reproduces the picture the literature reports: a source model validating near 0.99 macro-F_1 and scoring 0.16 on a second apartment, a drop that reads as severe domain shift. That measurement does not survive scrutiny. Features are computed over sliding windows with unit stride, so consecutive samples share 19 of their 20 timesteps, and a random

*Corresponding author.
kbaiermeryem@gmail.com (M. Kbaier);
nizar.bouguila@concordia.ca (N. Bouguila)

train/validation split places a 95%-identical copy of nearly every validation sample into training. We isolate the effect by varying the window definition and the split independently: the window moves source validation by under two points, the split moves it by forty-six. Grouping by recording day, a one-line change, gives 0.53 and leaves the target number essentially untouched. The gap that motivates this literature is therefore substantially smaller than reported, and the remainder of this paper measures both domains under the same protocol.

Once the protocol is fixed, the picture changes in a way that is more interesting than the original. Most of the remaining gap is attributable not to some irreducible property of cross-building transfer but to the features themselves. The conventional design devotes 106 of its 135 dimensions to per-sensor statistics, and those features are actively harmful across buildings: used alone they score below chance in both transfer directions, because `sensor_id_100` is a different physical device in each apartment. A model built from thirteen layout-invariant window statistics transfers far better while validating slightly worse in-domain.

With the features fixed, the question of what cross-building transfer actually costs can be put properly. Source-validation minus target performance, the quantity usually quoted, compares one building with another and so moves whenever the two differ in difficulty, whether or not anything transfers badly. Holding the evaluation set fixed and varying only where the training labels came from gives a different answer, and an asymmetric one. Into the second apartment, a model trained on the first scores within 0.6 points of a model trained on the second apartment's own days, at matched training-set size: there is essentially no cross-building penalty to speak of. Into the first apartment the same comparison costs 21.3 points. The price of changing building is not one number, and in one of these two directions it is close to zero.

That asymmetry organises everything that follows. Where there is no penalty there is nothing for adaptation to recover, and every method we test loses ground; where the penalty is real, the one method that can act within a short stream gains. The three methods otherwise divide cleanly. One helps a little. Two do not move at all, and in both cases the reason is specific, mechanical, and traceable to the original paper rather than to our implementation.

Our contributions are as follows.

- **A cross-building penalty measured against the right ceiling, and found to be asymmetric.** Holding the evaluation set fixed and matching training-set size, the penalty is 0.6 points entering one apartment and 21.3 entering the other (Section 6.3). Per-sensor features are what create the apparent gap: used alone they score below stratified-random on the target in both directions.
- **Adaptation gain measures source-model deficiency, not adaptation quality.** On cross-office occupancy estimation the two transfer directions differ only in the quality of the source checkpoint: against a sound one every method loses, against one worse than chance the same methods gain 28 to 43 MCC points. Across both tasks and both directions, sixteen measurements order by checkpoint quality alone (Section 6.11).
- **One mechanism, not four.** All four methods adapt batch-norm statistics, and each released implementation makes that its primary mechanism. With it disabled, PETAL and PeTTA change under 25 of 3412 predictions; with it enabled they land on AdaBN (Section 6.7). We also record three places where these papers and their own released code disagree, each in the direction of suppressing adaptation (Section 6.8).
- **A protocol error that accounts for most of the reported cross-building gap.** We isolate it by varying the window definition and the validation split independently. Causality and segmentation move source macro-F_1 by less than two points; the split moves it by forty-six (Section 6.1).
- **Causal features that are both deployable and better.** Windows centred on the labelled sample read five minutes into the future, contradicting the online premise of TTA. Replacing them with causal windows improves target macro-F_1 by 7.3 points. There is no trade-off to manage (Section 6.1).
- **A located ceiling.** Seven attempts to raise target performance beyond the deployable checkpoint are reported, six of which fail, including an unsupervised sensor-correspondence method given the advantage of offline access to the target stream (Sections 6.5 and 6.6).

Section 2 reviews domain adaptation and TTA for sensor data. Section 3 states the problem and the three methods. Section 4 describes our data handling, protocol and features. Sections 5 and 6 give the setup and results. Sections 7 and 8 discuss and conclude.

2. Related work

2.1. Domain adaptation in smart buildings

Cross-environment degradation in building sensor models is well documented, and the usual remedy is feature-level alignment using unlabelled target data gathered before deployment [4, 16, 5]. Open-set extensions handle activities present in the target but absent from source training [17, 18]. What these approaches share is an offline phase: target data must be collected and processed before the model goes live. A model arriving in a new building has none, annotating sensor streams is expensive, and taking a building management system offline to retrain is rarely acceptable. TTA exists to remove that phase.

2.2. Test-time adaptation and its assumptions

TTA adapts a pre-trained model during inference using only unlabelled target samples [9]. It is stricter than source-free domain adaptation [15], which permits repeated offline passes over the target set: TTA runs online, in a single pass, on samples as they arrive. TENT [13] minimises prediction entropy over batch normalisation parameters and assumes a stationary, i.i.d. test stream, both of which fail for building sensors, where distributions drift across the day and consecutive readings are correlated. Under class imbalance, entropy minimisation tends to reinforce the majority class, since confident low-entropy outputs are easiest to produce for frequent classes. CoTTA [14] extends TENT to continual settings with weight averaging and stochastic restoration, but relies on augmentation-averaged pseudo-labels, for which one-dimensional sensor data has no natural analogue.

The three methods we study were selected because their mechanisms are, at least in principle, modality-agnostic. DIGA [10] mixes source and instance normalisation statistics and adds a prototype classifier, computing no gradients at all. PETAL [11] places a posterior over source weights and derives a student–teacher objective with a source-posterior regulariser and Fisher-information-based restoration. PeTTA [12] targets recurring environments (Fig. 3), monitoring drift from the source model and modulating its adaptation rate to prevent collapse. Section 6.7 shows that each nonetheless embeds an assumption that fails here.

2.3. TTA for one-dimensional data

Outside vision, TTA is comparatively unexplored. Wang et al. [19] propose an optimisation-free method for wearable HAR using exponential-decay test-time normalisation. Related work studies source-free adaptation for EEG, which shares temporal correlation and channel heterogeneity with building sensors. None of this work addresses the combination we face: severe class imbalance, differing sensor layouts across sites, and recurring daily structure. Nor, to our knowledge, has any prior study examined how evaluation protocol and feature design interact with measured adaptation gains — which, as we show, dominates the comparison.

3. Background

3.1. Problem formulation

A domain is a pair $\mathcal{D} = (\mathcal{X}, P(X))$ with feature space $\mathcal{X}$ and input marginal $P(X)$. A model f_θ is trained on a source domain $\mathcal{D}_S = (\mathcal{X}_S, P_S(X))$ from labelled data $\{(\mathbf{x}_n, y_n)\}_{n=1}^N$ and deployed on a target $\mathcal{D}_T$ with $P_S(X) \neq P_T(X)$ and no labels. At step t the model receives $\mathbf{x}_t \in \mathcal{X}_T$, predicts $\hat{y}_t = f_{\theta_t}(\mathbf{x}_t)$, and updates $\theta_{t-1} \to \theta_t$ without seeing a label:

$$\min_\theta \; \mathbb{E}_{\mathbf{x}_t \sim P_T(X)} \big[\mathcal{L}_{\text{TTA}}(f_\theta(\mathbf{x}_t)) \big] + \mathcal{R}(\theta, \theta_0), \tag{1}$$

where θ_0 are the source parameters and $\mathcal{R}$ penalises drift.

Two constraints on Eq. 1 are usually left implicit and matter a great deal here. First, $f_{\theta_t}(\mathbf{x}_t)$ must be computable from information available at time t; a feature that reads future samples cannot be deployed. Second, any quantity used to define $\mathcal{X}$ (scalers, pruning masks, prototypes) must be estimable without target labels, and ideally without target data. Section 4 enforces both.

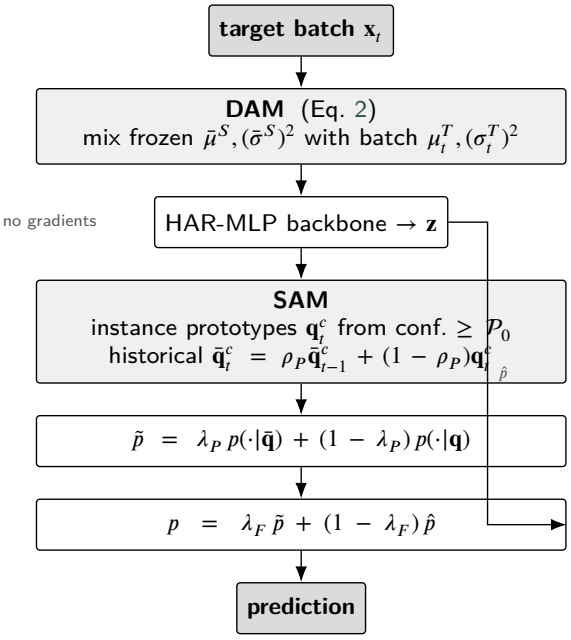


Figure 1: DIGA applied to the one-dimensional backbone. DAM mixes frozen source batch-normalisation statistics with the current batch's; SAM builds instance and historical prototypes and fuses their predictions with the parametric classifier. No gradient is computed anywhere in the loop.

3.2. DIGA

DIGA [10], summarised in Fig. 1, combines a Distribution Adaptation Module (DAM) and a Semantic Adaptation Module (SAM), with no gradient computation. DAM mixes frozen source batch-normalisation statistics with those of the current batch:

$$\begin{aligned} \bar{\mu}_t^T &= \lambda_{\text{BN}} \bar{\mu}^S + (1 - \lambda_{\text{BN}}) \mu_t^T, \\ (\bar{\sigma}_t^T)^2 &= \lambda_{\text{BN}} (\bar{\sigma}^S)^2 + (1 - \lambda_{\text{BN}}) (\sigma_t^T)^2. \end{aligned} \tag{2}$$

SAM estimates per-class prototypes $\mathbf{q}_t^c$ from predictions above confidence $\mathcal{P}_0$, carries a historical copy by exponential moving average, $\bar{\mathbf{q}}_t^c = \rho_P \bar{\mathbf{q}}_{t-1}^c + (1 - \rho_P)\mathbf{q}_t^c$, and builds *two* non-parametric classifiers which are mixed by λ_P before being fused with the parametric output by λ_F:

$$\tilde{p} = \lambda_P\, p(c|\mathbf{x}_t, \bar{\mathbf{q}}) + (1 - \lambda_P)\, p(c|\mathbf{x}_t, \mathbf{q}), \tag{3}$$

$$p = \lambda_F\, \tilde{p} + (1 - \lambda_F)\, \hat{p}. \tag{4}$$

We note that DIGA's Eq. 8 writes $p(c|\mathbf{x}, \mathbf{q}) \propto \exp(-\langle \mathbf{z}, \mathbf{q}_c \rangle)$. Read literally as an inner product this ranks the *least* similar prototype highest; we therefore treat $\langle \cdot, \cdot \rangle$ as a dissimilarity and report the effect of the alternatives in Section 6.7.

3.3. PETAL

PETAL [11], shown in Fig. 2, approximates the source posterior $q(\theta)$ by SWAG-diagonal and updates a student by

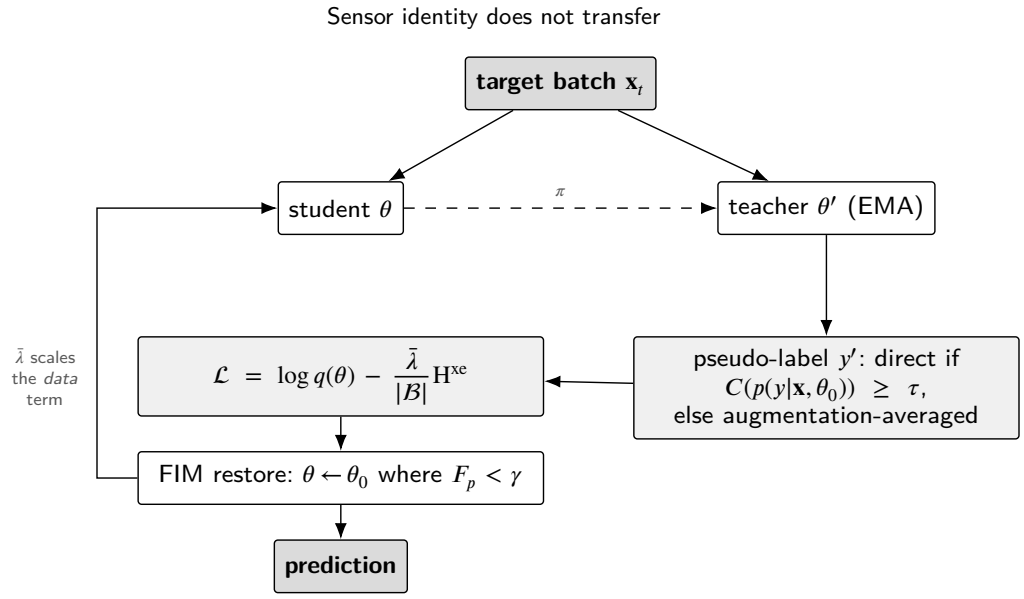


Figure 2: PETAL's student–teacher loop. The coefficient $\bar{\lambda}$ multiplies the cross-entropy, not the source-posterior term; Section 6.7 shows what follows when it is set to zero.

ascending

$$\mathcal{L} = \log q(\theta) - \frac{\bar{\lambda}}{|\mathcal{B}|} \mathrm{H}^{\mathrm{xe}}(y', y \mid \bar{\mathbf{x}}), \qquad \theta_{t+1} = \theta_t + \eta \nabla_\theta \mathcal{L}, \quad (5)$$

with pseudo-labels y' from an EMA teacher and a Fisher-information criterion restoring unimportant parameters to θ_0. The role of $\bar{\lambda}$ in Eq. 5 is worth stating explicitly, because it is easy to misread: $\bar{\lambda}$ multiplies the *data* term, and the prior weight is fixed at unity. Setting $\bar{\lambda} = 0$ does not remove a regulariser; it removes the likelihood, leaving an update that does not depend on the target stream at all.

3.4. PeTTA

PeTTA [12] targets recurring environments. It senses divergence in feature space through a Mahalanobis distance between per-class running means and their source counterparts,

$$\gamma_t^y = 1 - \exp\big(-(\hat{\mu}_t^y - \mu_0^y)^\top (\Sigma_0^y)^{-1} (\hat{\mu}_t^y - \mu_0^y) \big), \quad (6)$$

and uses the batch average $\bar{\gamma}_t$ to set both the regularisation weight and the teacher update rate,

$$\lambda_t = \bar{\gamma}_t \lambda_0, \qquad \alpha_t = (1 - \bar{\gamma}_t) \alpha_0. \quad (7)$$

The quadratic form in Eq. 6 is a *sum* over embedding dimensions. Section 6.7 shows what that implies when the two domains are different buildings rather than corruptions of the same image.

4. Methodology

4.1. Windowing

Binary sensor activations are aggregated over windows of $W = 20$ consecutive timesteps at 30 s resolution, giving a 10-minute window per labelled sample. Two choices in this step are usually made implicitly, and we make both explicit because they turn out to matter.

Causality. A window may be *centred* on the labelled sample, spanning $[i - W/2, i + W/2]$, or *causal*, spanning $[i - W + 1, i]$. The centred form is common and reads five minutes into the future of every sample, which is incompatible with online deployment. We use causal windows throughout and quantify the difference in Section 6.1.

Segmentation. Windows must be truncated somewhere. Segmenting on *ground-truth activity changes* guarantees that no window straddles two activities, but hands the extractor the label transitions it is meant to predict. We segment instead on *recording continuity*: a new segment begins at a day change or a gap in the 30 s sampling. This uses no labels and is what a deployed system can determine. Windows are truncated at segment starts instead of dropped, which is what an online system genuinely has after a gap.

4.2. Validation protocol

Because stride is one, adjacent samples share $W - 1$ of W timesteps. A random train/validation split therefore places a near-duplicate of nearly every validation sample into training, and the resulting score measures memorisation, not generalisation. We use **GroupKFold over `year_day`**: no recording day appears in both halves, which removes window overlap and same-day correlation together. All source model selection in this paper uses this protocol. Section 6.1 reports what each alternative gives.

4.3. Features

The conventional 135-dimensional vector comprises 106 per-sensor statistics (mean and standard deviation for each of the 53 shared sensors), five window-level activity statistics, 20 co-activation indicators for the most frequent sensor pairs in the source, and four cyclic time encodings. Of these, 126 are tied to *sensor identity*: the index 100 refers to a different physical device in each building, so a rule learned about it is not merely uninformative in the target but wrong.

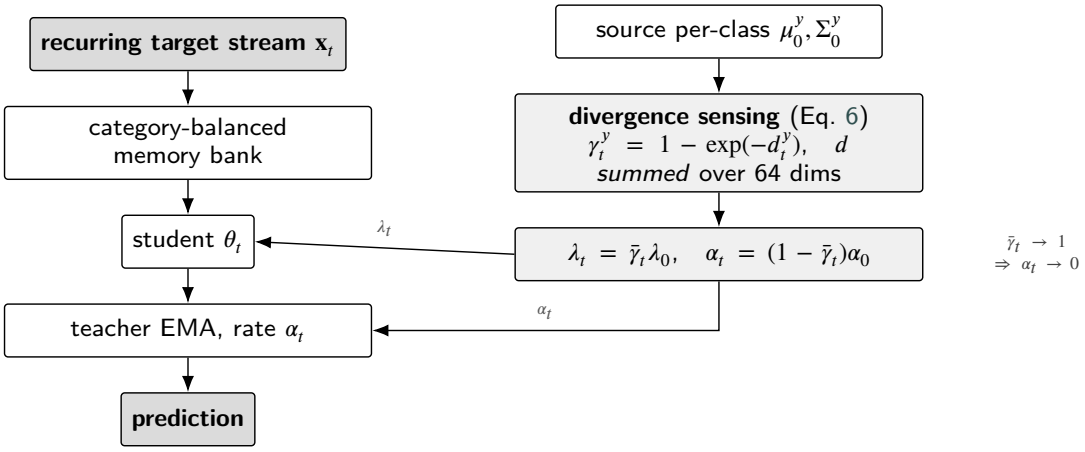


Figure 3: PeTTA for recurring streams. A single scalar $\bar{\gamma}_t$, derived from a Mahalanobis distance *summed* over the embedding dimension, sets both the regularisation weight and the teacher update rate. As $\bar{\gamma}_t \to 1$ the update rate goes to zero.

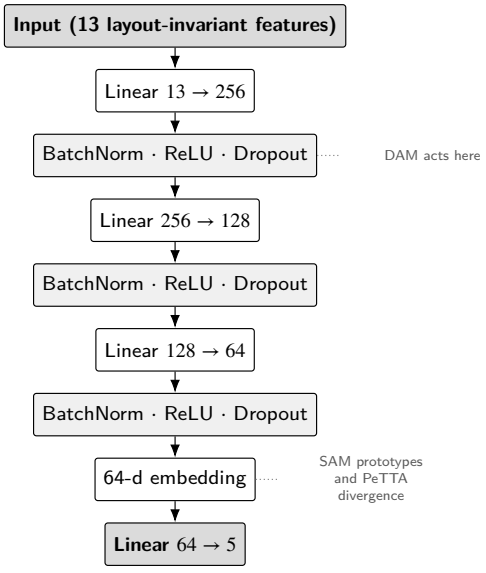


Figure 4: The classifier used throughout. Only the input width changes between feature sets; depth, widths and regularisation are held fixed so that comparisons isolate the features.

We therefore evaluate a 13-dimensional **layout-invariant** set: nine window-level statistics of the per-timestep active-sensor count (mean, maximum, minimum, median, standard deviation, idle fraction, activation entropy, transition count, transition rate) and four cyclic encodings of time of day and day of week. Nothing in this set refers to which sensor fired. Appendix A reports a larger invariant set including a sorted activation profile and run-length structure; it does not improve on the thirteen.

Fig. 5 projects the engineered space onto its first two source principal components; the target occupies a displaced, compressed region.

All transforms are fitted on the source training split only. No target labels and no target statistics are used at any point in constructing the source model, so every configuration remains a legal TTA starting point.

4.4. Causal prediction smoothing

Activities persist for minutes while predictions are made per 30 s window. We apply a median filter over the last L predictions, never crossing a recording-segment boundary. This uses only past outputs and so remains deployable online. We report $L = 41$, selected on source data.

4.5. Stream protocol

Target data is divided into eight contiguous segments. *Non-recurring* streams visit each once; *recurring* streams make three passes in randomised order, repeated over three independent orderings. Target labels are withheld throughout adaptation and used only to score predictions afterwards.

4.6. What a method is allowed to know

Methods in this literature differ in what they require of the target building before they can produce a prediction, and the distinction is usually left implicit. We separate three levels.

Tier 1, no target data. Everything is estimated on the source. The model can be installed in a new building and run immediately. This is the setting test-time adaptation exists to serve.

Tier 2, unlabelled target data, offline. The full target distribution must be collected before deployment, though no labels are needed. This is source-free domain adaptation. It is a weaker requirement than labels but it reintroduces exactly the data-collection phase that motivates TTA.

Tier 3, target labels. Not deployable.

The distinction matters for what follows. The layout-invariant features used throughout this paper are tier 1: every transform is fitted on the source split alone, and the model needs nothing from the target building. Feature-shift pruning, by contrast, selects features by comparing source and target distributions, and is therefore tier 2 — unsupervised, but not deployable without a prior observation period. The hyperparameter selection discussed next is tier 3.

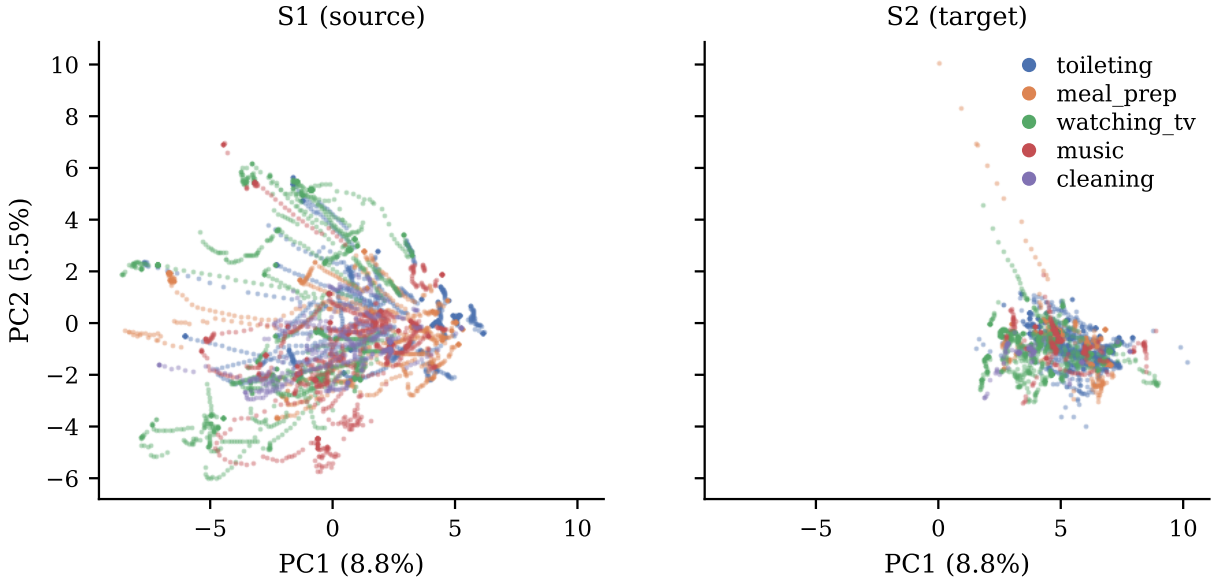


Figure 5: First two principal components of the causal engineered feature space, fitted on S1 and applied unchanged to S2. The target occupies a displaced and markedly compressed region, which is the shift the adaptation methods are asked to absorb.

4.7. Hyperparameter selection

Two evaluation protocols, and which number belongs to which. Results that compare *configurations* use the full protocol: GroupKFold over `year_day`, 3 seeds × 5 folds, causal median smoothing at $L = 41$, scored on the whole target stream. Source-only there is macro-F_1 0.4466 (S1→S2). Results that compare *adaptation methods* run a single stream over a single source checkpoint, since a streaming method cannot be averaged over folds without restarting its state; those use the last GroupKFold fold, no smoothing, and source-only is 0.4944 averaged over the same three seeds. The two differ because smoothing adds about 1 point, fold choice a little more, and the single-split source model has a seed spread of 0.027. Tables 14 and 16 predate the three-seed convention and quote the seed-0 value of that same checkpoint, 0.4641; we give both so no figure in this paper is unattributable to a protocol. Comparisons are always within a protocol, never across.

We report results using each method's *published* hyperparameters, and separately under target-label tuning where the two differ materially. Selecting on target labels is tier 3 by the distinction above and is not available in deployment. It is also not merely optimistic: Section 6.7 shows it selecting a configuration at which one method cannot adapt at all, and reporting the maximum of a distribution that is mostly negative for another. Where a label-free criterion is needed we use mean prediction entropy on the target stream, and report what it selects alongside the oracle value. So that no conclusion rests on a single setting, Section 6.10 searches each method's full published hyperparameter grid on the activity task in both directions, and Section 6.7 does the same for DIGA on occupancy.

5. Experimental setup

5.1. Datasets

HAR. The dataset of Tapia et al. [20] provides two single-occupant apartments, S1 and S2, instrumented with binary proximity sensors on household objects and recorded over roughly two weeks each (16 recording days here). S1 exposes 70 sensor columns and S2 exposes 75; feature engineering uses the 53 shared by both. Table 1 gives per-class counts. We evaluate both transfer directions.

Class names follow the source annotation (`toileting`, `meal_prep`, `watching_tv`, `music`, `cleaning`). We note this because the time-of-day distributions confirm them: activity code 65 occurs between 07:31 and 14:51 with a median of 11:34, which is consistent with `watching_tv` and not with a sleep-type label.

Occupancy. A private dataset collected from two university offices (H355 and H358) at the Grenoble Institute of Technology using CO_2 concentration sensors [6, 5]. The original annotation records occupancy counts (zero, one, two, three, more than three); we binarise to occupied / not occupied. The representation is nine engineered features produced by the originating pipeline; we use them as supplied and perform no further feature construction, so no target information enters through any step we control. Splits are 1145 / 200 source train / test and 822 / 150 target train / test, with roughly 78% / 22% class balance. Because the data is private, the released code reproduces the HAR results only.

Table 1
HAR dataset statistics. Class names follow the source annotation.

Split	Total	toileting	meal_prep	watching_tv	music	cleaning
S1	5031	1065	857	1598	1069	442
S2	3412	1266	417	1291	325	113

Table 2
Source-domain validation macro-F_1 by windowing policy and validation protocol (135 features, z-score, mean ± s.d. over 5 seeds). Reading down any column shows that the window definition is nearly irrelevant; reading across any row shows that the split is not.

Causality	Segmentation	Random 80/20	Blocked	Grouped by day
centred	activity labels	0.9984 ± 0.0009	0.4357 ± 0.0345	0.5287 ± 0.0067
centred	recording	0.9979 ± 0.0006	0.4571 ± 0.0451	0.5258 ± 0.0112
causal	activity labels	0.9975 ± 0.0021	0.4414 ± 0.0131	0.5261 ± 0.0084
causal	recording	0.9968 ± 0.0022	0.4608 ± 0.0209	0.5110 ± 0.0115

5.2. Model

A three-layer MLP with batch normalisation, ReLU and dropout (Fig. 4), reducing the input to a 64-dimensional embedding followed by a five-class output layer. The architecture is held fixed across all feature sets so that comparisons isolate the features.

5.3. Metrics and baselines

We report accuracy, macro-F_1, MCC [21] and per-class F_1. macro-F_1 is the primary metric for HAR because it weights all activities equally, which matters when the target minority classes are small.

Chance baselines are reported in every table and are not optional here: several published figures for this transfer setting sit at or below them. On S2, constant-majority prediction gives accuracy 0.378 and macro-F_1 0.110, and stratified-random prediction gives accuracy 0.306 and macro-F_1 **0.200**. On S1 the stratified-random macro-F_1 is 0.200. For binary occupancy, always predicting "not occupied" gives accuracy 0.773, macro-F_1 0.436 and MCC 0.000.

Differences between methods are reported with 95% paired bootstrap confidence intervals over the target stream (2000 resamples), computed on the difference from source-only.

6. Results

6.1. Decomposing the source-domain fit

Establishing what the cross-building gap actually is requires measuring both domains under one protocol. Table 2 and Fig. 6 vary the window definition and the validation split independently, holding the feature arithmetic fixed, which separates the contribution of each.

A fully causal pipeline with no label-derived segmentation still validates at 0.9968 under a random split. The inflation is therefore attributable to window overlap under random splitting, not to the window definition. Grouping by recording day gives 0.51–0.53, and we treat that as the honest source-domain figure.

Causality nonetheless matters, in the opposite direction to what one might expect. Under a fixed training protocol, making features causal *improves* target transfer (Table 3): macro-F_1 rises from 0.1626 to 0.2356, with accuracy rising from 0.375 to 0.444. The deployable pipeline is also the better-transferring one, so nothing is traded away by adopting it.

6.2. Feature design

Table 4 sweeps feature subsets and source-fitted transforms on causal, recording-segmented features under GroupKFold selection.

Two observations. Per-sensor features alone reach 0.1484 macro-F_1, which is 5.1 points *below* stratified-random guessing. They are not weakly informative; they encode rules that are wrong in the target building. And the ordering by source validation is not the ordering by transfer: the best-validating row (KS-pruned with rank-gauss, 0.6232) transfers at 0.3004, while the best-transferring row (0.4133) validates 8.9 points lower. Across the eight rows the two columns are still positively related ($\rho = +0.69$), so this is a reordering near the top and not a reversal; what it costs a practitioner is quantified in Section 6.3.

Table 5 and Fig. 7 repeat the comparison in both directions.

The below-chance result replicates: per-sensor features fall short of stratified-random guessing in both directions. The reordering is sharper in reverse, where the conventional 135-d set has the best source-validation score of any row (0.5381) and the worst transfer (0.1583), itself 4.2 points below chance. Choosing between these four feature sets by source validation alone would select that row.

The gap is direction-dependent: 8.8 points forward and 14.7 reverse. The asymmetry has a visible cause rather than being noise. S2 is the weaker source: 3412 samples against 5031, with the hard classes much thinner (113 versus 442 for `cleaning`) — and training on the smaller, more imbalanced apartment lowers both sides of the comparison. We therefore

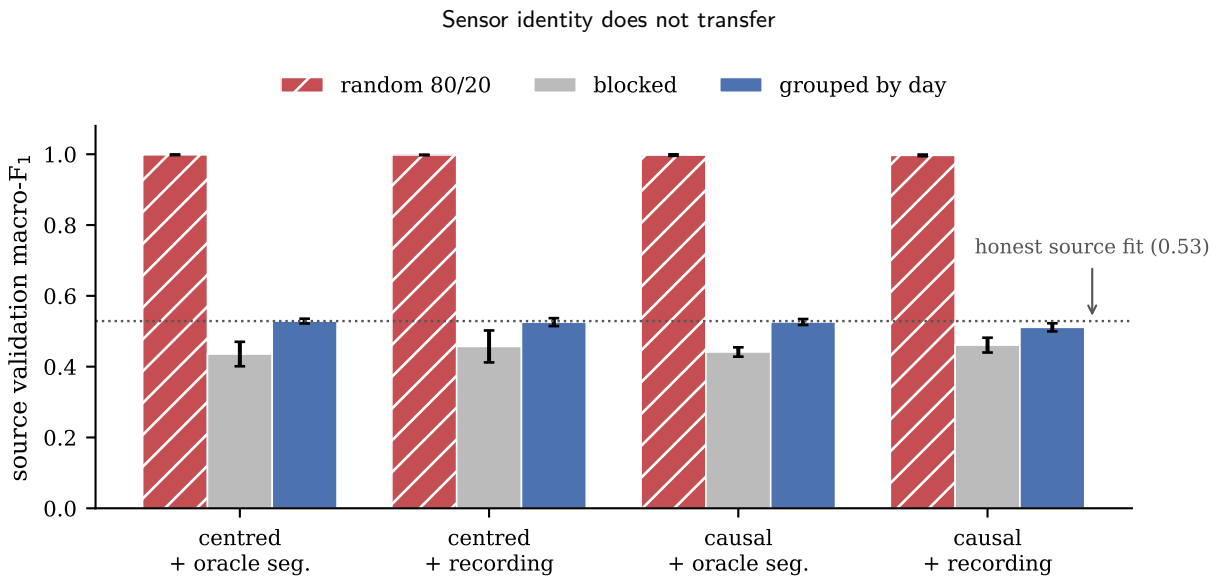


Figure 6: Source validation macro-F_1 under every combination of windowing policy and validation split (mean ± s.d. over 5 seeds). The hatched bars are the random split. Its height is independent of how the window is defined, which locates the inflation in the split rather than in the features.

Table 3
Target transfer (S1→S2) by windowing policy, 135 features, blocked-trained, mean over 5 seeds. Stratified-random macro-F_1 is 0.200.

Causality	Segmentation	macro-F_1	MCC	Accuracy
centred	activity labels	0.1626 ± 0.0125	0.0945	0.3753
centred	recording	0.1953 ± 0.0274	0.1327	0.3924
causal	activity labels	0.2344 ± 0.0232	0.2036	0.4533
causal	recording	$\mathbf{0.2356} \pm 0.0124$	0.1805	0.4440

Table 4
S1→S2 transfer by feature subset and source-fitted transform (causal features, GroupKFold over 16 days, 5 seeds). Stratified-random macro-F_1 is 0.200.

Feature set	Transform	d	Source-val macro-F_1	Target macro-F_1	Target MCC
All (conventional)	z-score	135	0.5083	0.2436	0.1682
All	rank-gauss	135	0.5890	0.3775	0.3144
KS-pruned 20%	z-score	108	0.5164	0.3130	0.2090
KS-pruned 20%	rank-gauss	108	**0.6232**	0.3004	0.2138
No per-sensor	z-score	29	0.5121	0.3561	0.2633
Global + temporal	z-score	9	0.5341	**0.4133**	**0.3490**
Global only	z-score	5	0.2914	0.2129	0.0970
Per-sensor only	z-score	106	0.3292	0.1484	0.0436

Table 5
Forward and reverse transfer under an identical protocol (causal features, GroupKFold over the source's days, causal smoothing $L = 41$). "Gap" is source-validation minus target macro-F_1, in points.

Direction	Feature set	d	Source-val	Target macro-F_1	MCC	Gap	vs. chance
S1→S2	Layout-invariant	13	0.5341	**0.4466**	0.3973	8.8	+24.7
S1→S2	All (conventional)	135	0.5133	0.2118	0.0976	30.2	+1.2
S1→S2	Per-sensor only	106	0.3248	0.1350	0.0123	19.0	**−6.5**
S1→S2	No per-sensor	29	0.5119	0.3630	0.2734	14.9	+16.3
S2→S1	Layout-invariant	13	0.4538	**0.3068**	0.2667	14.7	+10.7
S2→S1	All (conventional)	135	**0.5381**	0.1583	0.0332	38.0	**−4.2**
S2→S1	Per-sensor only	106	0.3668	0.1702	0.0553	19.7	**−3.0**
S2→S1	No per-sensor	29	0.4483	0.2271	0.1043	22.1	+2.7

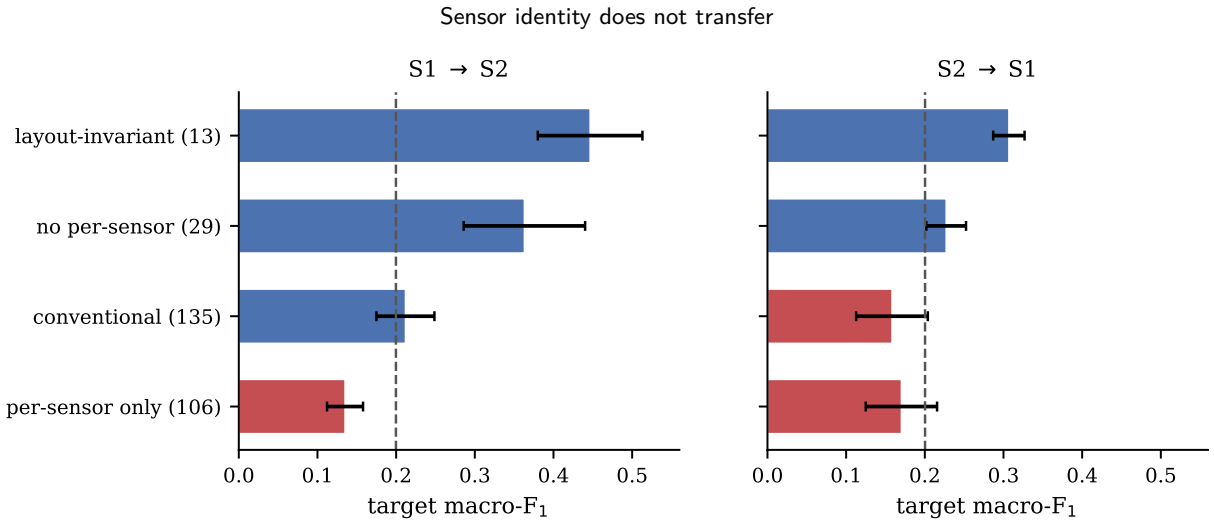


Figure 7: Target macro-F_1 by feature set in both transfer directions, with the stratified-random baseline marked (dashed). Bars falling short of it are shown in red. Per-sensor features are below chance in both directions; the conventional 135-d vector is below chance in one of them.

Table 6
Cross-building penalty on a fixed evaluation set at matched training-set size, 3 seeds × 5 folds. *Own days* trains on the target apartment; *other building* trains on the source apartment. The difference is what a deployment pays for having no labels in the new building.

Evaluated on	Own days	Other building	Penalty
S2 held-out days	0.3497	0.3434	**0.6** pts
S1 held-out days	0.4897	0.2767	21.3 pts

state the result as a gap of 9 to 15 points instead of quoting the more favourable direction.

6.3. What does changing building actually cost?

Table 5 reports source-validation minus target macro-F_1 and calls it a gap, as is conventional. That quantity compares performance on one building with performance on another, so it moves whenever the two differ in difficulty, whether or not anything transfers badly. S1 and S2 do differ: 5031 samples against 3412, and 442 `cleaning` windows against 113. Part of the 8.8 and 14.7 points is the buildings, not the transfer.

The quantity that isolates domain shift holds the evaluation set fixed and varies only where the training labels came from. For each apartment in turn we train one model on that apartment's own days and another on the other apartment entirely, and score both on the same held-out days under GroupKFold, with identical features, folds and smoothing. Training-set sizes are matched by subsampling, because S2 has two thirds of S1's data and an unmatched comparison would charge domain shift for a difference in sample count.

The penalty is not one number. Entering S2 it is 0.6 points, which is well inside the seed spread of either model and is not distinguishable from zero. A model that has never seen S2 performs as well on S2 as a model trained on S2's own days. Entering S1 the same comparison costs 21.3 points.

Two things follow, and both matter for how the rest of this paper should be read.

The first is that the conventional gap figure overstates the forward direction badly. The 8.8 points of Table 5 are almost entirely the difference between the two apartments as prediction problems, not the cost of moving between them. Reported cross-building drops that take this form should be read the same way.

The second is that one of our two transfer directions contains no domain shift for a method to correct. This is not a limitation of the experiment but the most useful thing in it: it provides a control. A test-time adaptation method run on S1→S2 is being asked to improve a model that is already performing at its target ceiling, and any movement it produces can only be downward. Run on S2→S1 the same method has 21 points of genuine headroom. Section 6.10 shows the published methods behaving exactly along that line, and Section 6.11 reproduces the same split on a second task.

Does in-domain validation predict transfer? The asymmetry above raises the related question of whether source-domain fit is a usable guide when choosing what to deploy. Pooling every configuration in this study for which we have a matched source-validation and target score (70 in total, across feature sets, transforms, window lengths, classifier families and temporal models), the rank correlation is +0.63 ($p < 0.001$). In-domain validation is therefore positively informative overall, and we make no claim to the contrary. Among configurations that are actually competitive, however, it degrades: restricting to those validating at 0.48 or above leaves $\rho = +0.27$ ($p = 0.11$), and the single best-validating configuration is not the best transferring one in either direction of Table 5. The honest summary is that in-domain validation separates working configurations from broken ones and carries little information beyond that, which

Table 7
TTA on the layout-invariant causal checkpoint (3 seeds; recurring streams additionally averaged over 3 randomised segment orderings). Stratified-random macro-F_1 is 0.200.

Method	Stream	Accuracy	macro-F_1	MCC
Source-only	non-recurring	0.6057 ± 0.012	0.4481 ± 0.027	0.4535 ± 0.031
AdaBN	non-recurring	0.3784 ± 0.012	0.2992 ± 0.004	0.1526 ± 0.014
TENT	non-recurring	0.3781 ± 0.011	0.2986 ± 0.005	0.1519 ± 0.013
DIGA	non-recurring	0.6028 ± 0.013	$\mathbf{0.4618} \pm 0.004$	$\mathbf{0.4720} \pm 0.006$
PETAL	non-recurring	0.3622 ± 0.005	0.2771 ± 0.015	0.1220 ± 0.011
Source-only	recurring	0.6057 ± 0.010	0.4481 ± 0.023	0.4535 ± 0.026
DIGA	recurring	$\mathbf{0.6060} \pm 0.010$	0.4603 ± 0.016	0.4676 ± 0.024
PETAL	recurring	0.3640 ± 0.008	0.2771 ± 0.006	0.1215 ± 0.010
PeTTA	recurring	0.3721 ± 0.008	0.2848 ± 0.007	0.1358 ± 0.011

Table 8
95% paired bootstrap confidence intervals on the difference from source-only (2000 resamples, same stream).

Method	Stream	Δmacro-F_1	ΔMCC
DIGA	non-recurring	[−0.0025, +0.0177]	[+0.0215, +0.0500]
DIGA	recurring	[+0.0124, +0.0289]	[+0.0361, +0.0610]
PETAL	non-recurring	[−0.1990, −0.1619]	[−0.3533, −0.2978]
PETAL	recurring	[−0.1948, −0.1574]	[−0.3451, −0.2899]
PeTTA	recurring	[−0.1889, −0.1515]	[−0.3278, −0.2696]
AdaBN	non-recurring	[−0.1725, −0.1350]	[−0.3029, −0.2476]
TENT	non-recurring	[−0.1745, −0.1374]	[−0.3050, −0.2503]

is the regime a practitioner choosing between reasonable alternatives is in.

6.4. Test-time adaptation on the deployable checkpoint

Table 7 compares the methods on the deployable checkpoint.

In Table 8, DIGA is the only method whose interval excludes zero on the positive side, and the effect is small: +1.7 macro-F_1 and +2.2 MCC points. It is also contingent on how two under-specified steps of the method are implemented, which Section 6.9 quantifies. For comparison, the feature redesign is worth +17.0 macro-F_1 points before any adaptation runs. PETAL and PeTTA are statistically indistinguishable from the unadapted model. AdaBN and TENT are substantially *worse*, by around 17 points of macro-F_1, which is itself informative: adapting normalisation statistics helps when the source model is poor and hurts once the features are already distribution-robust.

Per-class F_1 (Table 9) shows four of five classes working. DIGA's prototype correction does what it is designed to do, lifting `meal_prep` by eight points at a cost of seven on `toileting`. `cleaning`, with 113 target samples, remains near zero for every method.

6.5. What does not move the result

Target performance is bounded by source performance: the deployable checkpoint validates at 0.534 in-domain and transfers at 0.447, a gap of under nine points. Raising the target therefore requires raising the source with features that still cross buildings. We report seven attempts, all negative, because together they locate the ceiling.

Two things are worth drawing out of Table 10.

First, the only attempt that helps in *both* transfer directions is the transition model, and it helps by less than a point (0.4466 → 0.4503 forward, 0.3068 → 0.3160 reverse). Its transition matrix is estimated on the source apartment and encodes nothing about sensors, only how often one activity follows another. That it transfers at all, while every attempt to extract more from the sensor stream does not, is consistent with the rest of this paper: what crosses buildings is behavioural regularity, not layout.

Second, and more importantly, several rows raise the source-validation score while lowering the target. Window length does (0.5611 at W=60 against 0.5341 at W=20), and so does temporal context (0.5414 for the TCN). Section 6.2 observed the same thing across feature sets. The pattern is general, and Section 7 returns to it.

6.6. Can the sensors be matched across buildings?

If per-sensor features fail only because index 100 denotes a different device in each apartment, the obvious repair is to estimate the correspondence. We tested this. Each sensor was described by a behavioural signature computed from unlabelled data: firing rate, burst-length statistics, a 24-bin diurnal profile, and an index-free summary of its co-activation neighbourhood. The two sensor sets were then matched by balanced assignment on signature distance.

Table 9
Per-class F_1 on S2 (mean over seeds and orderings).

Method	toileting	meal_prep	watching_tv	music	cleaning
Source-only	0.551	0.493	0.746	0.435	0.009
DIGA	0.478	**0.572**	**0.766**	**0.468**	0.035
PETAL	0.550	0.474	0.746	0.440	0.014
PeTTA	0.552	0.494	0.746	0.434	0.009
AdaBN	0.365	0.310	0.482	0.275	0.056

Table 10
Attempts to raise target macro-F_1 beyond the 13-d causal checkpoint (S1→S2, same protocol throughout). Only causal smoothing and a source-estimated transition model help, and both marginally.

Attempt	What was varied	Source-val	Target macro-F_1
baseline	13-d, MLP, W=20, median L=41	0.5341	**0.4466**
richer invariant features	+ sorted profile, runs, multi-scale (60-d)	–	0.4188
class weighting	inverse-frequency weights	–	0.3852
network width	64–32 and 128–64	–	0.4297
classifier family	boosting, forests, linear	0.5230	0.4190
window length	$W \in \{10, 40, 60\}$	0.5611	0.4330
temporal context	stacked T=11, causal TCN T=21	0.5414	0.4111
sensor correspondence	unsupervised assignment (§6.6)	–	0.2227
transition model	source-estimated HMM, causal filtering	0.5341	0.4503

Table 11
Per-sensor features under four permutations, target macro-F_1. A correspondence that is real should transfer between halves of the target stream.

Permutation	Fitted half	Held-out half
identity (no matching)	0.0976	0.1744
learned signatures	0.0749	0.2348
greedy search on fitted half	**0.4174**	0.1738
random, 20 draws	—	0.1655 ± 0.050

To separate a real correspondence from overfitting, permutations were estimated on one half of the target stream and scored on the other.

A greedy search reaches 0.417 on the half it is fitted to and 0.174 on held-out data, indistinguishable from identity and from random. The signature-based assignment is inconsistent between halves. No permutation we could find transfers.

We note that this test is permissive: estimating a correspondence from the whole unlabelled target stream is offline source-free adaptation, not test-time adaptation (Section 4.6), so it has an advantage no deployable method would have. Failing under relaxed assumptions implies failing under strict ones.

The conclusion is stronger than the premise it tests. Per-sensor features do not fail because they are mislabelled across buildings. They fail because a sensor in one apartment has no counterpart in the other: rooms differ in shape, furniture sits elsewhere, and routines differ. Relabelling cannot recover information that was never shared.

6.7. Three mechanisms

A natural objection to what follows is that these methods were designed for images, so their failure on one-dimensional data is to be expected. The results do not support that reading. The methods are not uniformly unhelpful: against the weakest source checkpoint in this study every one of them gains between 27 and 43 MCC points (Table 20). What varies is not the modality but the checkpoint they are given, and the mechanism that produces both outcomes is the same one.

All four methods adapt batch-normalisation statistics, and in each released implementation that is the primary mechanism rather than an incidental detail. AdaBN replaces the source estimates with the test batch's outright. DIGA blends them, retaining a fraction λ_{BN} of the source estimate (Eq. 2). PETAL's `configure_model` sets `track_running_stats` to false and discards `running_mean` and `running_var` on every normalisation layer, and because its EMA teacher is built by copying the configured model, the teacher that produces PETAL's predictions runs on batch statistics too. PeTTA installs a normalisation layer that tracks the test stream. In each case the gradient machinery the method is named for sits on top of that channel.

Separating the two is therefore the first thing to do, and Table 12 does it by running each method twice: once as its own code configures it, and once identically except that the source statistics are left in place.

Two readings follow, and the second is the one that matters.

With the channel disabled, PETAL and PeTTA are inert: between 5 and 24 altered predictions out of 3412, in either

Table 12
Each method with its normalisation channel active, as its released code configures it, and with that channel disabled and everything else unchanged. Δ is against source-only in the same direction; *changed* counts altered predictions out of 3412. The cross-building penalty is 0.6 points entering S_2 and 21.3 entering S_1 (Section 6.3).

	$S_1 \to S_2$			$S_2 \to S_1$		
Method	macro-F_1	Δ	changed	macro-F_1	Δ	changed
Source-only	0.4944	—	0	0.3226	—	0
AdaBN	0.3139	−18.0	1784	0.2137	−10.9	3143
TENT	0.3141	−18.0	1782	0.2133	−10.9	3142
DIGA	0.4088	−8.6	1279	**0.3438**	**+2.1**	1031
PETAL	0.3113	−18.3	1739	0.2028	−12.0	3105
PeTTA	0.2908	−20.4	1845	0.2158	−10.7	3164
normalisation channel disabled, everything else identical						
PETAL	0.4947	+0.0	11	0.3224	−0.0	24
PeTTA	0.4947	+0.0	5	0.3225	−0.0	13

Table 13
DIGA as λ_{BN} moves from source statistics (1.0) to target statistics (0.0), Δmacro-F_1 against source-only. $\lambda_{BN} = 1.0$ leaves the statistics untouched, so that row isolates the prototype classifier from the normalisation.

λ_{BN}	$S_1 \to S_2$	$S_2 \to S_1$
1.0 (source)	−5.7	**+4.4**
0.9	−6.5	+2.4
0.8 (published)	−8.6	+2.1
0.6	−23.3	−12.9
0.4	−30.3	−19.9
0.2	−25.9	−22.8
0.0 (target)	−33.0	−20.5

direction. Their gradient updates, their posterior regularisation, their divergence gating and their memory banks together move almost nothing. With the channel active, PETAL lands within 0.3 points of AdaBN in the forward direction and PeTTA within 1.1, having each changed roughly the same predictions AdaBN changes. Whatever these methods contribute over plain batch-statistic replacement, it is not visible at this scale. That is a sharper statement than reporting that they do not help, and it does not depend on any choice we made: it is what their own configuration does.

The channel itself is a continuum rather than a switch, and DIGA's λ_{BN} exposes it directly. Table 13 sweeps it from pure source statistics to pure target statistics.

The cost rises monotonically with the share of target statistics mixed in, in both directions, and it is steep: moving from $\lambda_{BN} = 0.8$ to 0.6 costs 15 points forward and 15 reverse. Note also what the top row says. At $\lambda_{BN} = 1.0$ the statistics are exactly the source model's, yet 1026 predictions change, because DIGA's prototype classifier is still active. That row is the only configuration anywhere in this study where an adaptation component helps by more than a point without target labels, and it helps in the direction with headroom and hurts in the direction without.

So DIGA's small positive result is not normalisation adaptation succeeding where the others failed. It is a prototype classifier gaining 4.4 points, partly spent back by a DAM that costs 2.3. The component of these methods that transfers to this data is the one that does not touch normalisation statistics at all.

On implementation: each method is implemented from the equations and hyperparameters given in its source paper, checked against its released code, and run at published settings; we report retuned values separately where they differ materially. Section 6.8 records where the papers and their released implementations disagree, since three of those disagreements change results here.

PETAL: the coefficient scales the likelihood. As noted in Eq. 5, $\bar{\lambda}$ weights the data term. Table 14 sweeps it on the deployable checkpoint with a non-degenerate SWAG posterior.

The consequence is specific. On a source model that already outperforms what self-training produces, selecting $\bar{\lambda}$ by target-domain macro-F_1, a criterion in common use, prefers $\bar{\lambda} = 0$, the setting at which adaptation cannot occur. A reported null for PETAL obtained this way is a statement about the search procedure, not about the method.

Two qualifications belong with that result. The first is that $\bar{\lambda} = 0$ is not the setting PETAL's authors run. Algorithm 2 as printed places the coefficient on the data term with the prior fixed at unity, which is what Eq. 5 and Table 14 implement; the released code writes the same objective the other way up, as `loss_H + spw·para_loss` with `spw` $= 10^{-9}$, placing the coefficient on the prior and fixing the data term at unity. The two are equivalent up to scale, and the published `spw` corresponds to $\bar{\lambda} \approx 10^9$: the opposite end of the range from the value target-label selection prefers. Our point is about what the selection rule does, not about what the authors run, and it should be read that way.

The second is that $\bar{\lambda}$ is not why PETAL is inert in Table 14 at all. The predictions barely move at any value, including $\bar{\lambda} = 10$, because that table was produced with

Table 14
PETAL as a function of $\bar{\lambda}$ (published parameterisation, Eq. 5). Source-only is macro-F_1 0.4641, MCC 0.4679. At $\bar{\lambda} = 0$ the weights still move, toward the SWAG mean, but the movement is independent of the target stream.

$\bar{\lambda}$	Drift from source	Predictions changed	macro-F_1	MCC
0.0	1.0021	24/3412	0.4676	0.4724
0.1	0.8118	26/3412	0.4650	0.4687
1.0	0.3580	17/3412	0.4643	0.4679
10.0	0.1688	22/3412	0.4637	0.4677

Table 15
PeTTA divergence sensing on S2, with the normalisation channel of Table 12 disabled so the gating is visible in isolation. The released implementation uses the per-dimension mean.

Reduction	mean $\bar{\gamma}$	α_t	mean d	Changed
Sum over embedding dimensions	1.000000	1.2×10^{-10}	71.8	0/3412
Per-dimension mean (released)	0.5603	4.4×10^{-4}	1.12	10/3412
Sum + α floor	1.000000	1.0×10^{-3}	71.6	1/3412

the normalisation channel of Table 12 disabled. With the channel active, as PETAL's `configure_model` sets it, PETAL moves 1739 predictions and loses 18.3 points. The $\bar{\lambda}$ sweep therefore measures what the gradient term alone contributes, which is almost nothing, and should not be read as a statement about the method as published.

The same selection rule distorts DIGA, in the opposite direction. Running its full published grid (Section 3, 64 configurations) on the occupancy task, every one of the 64 configurations *reduces* MCC relative to source-only once averaged over the three seeds: the median costs 1.0 point and the least damaging costs 0.7. Across individual runs, 83% are negative and the single best run gains 0.9 points, so a search reporting its own maximum over a grid and a seed can return a positive number drawn from a distribution with no positive mean. Selection on target labels returns exactly that maximum. A label-free rule, lowest mean prediction entropy on the target stream, selects the configuration costing 0.7 points, which is what a deployment would actually obtain. Section 6.10 repeats the exercise on the activity task and for all three methods.

PeTTA: a reduction that is easy to get wrong. Eq. 6 is written as a quadratic form, which reads as a sum over the 64-dimensional embedding. We implemented it that way, and on this data that choice alone decides whether the method runs at all. Summed, the cross-building statistic averages 71.8 over the stream, $\exp(-d)$ underflows, $\bar{\gamma}$ pins at unity, and Eq. 7 sets α_t to 1.2×10^{-10}: the teacher stops. Averaged over the same dimensions, the statistic is 1.12, $\bar{\gamma}$ is 0.56, and α_t recovers to 4.4×10^{-4}.

PeTTA's released implementation averages. Its divergence is a mean over the embedding dimension, so the saturation above is a property of our reduction and not of the published method, and we report it as such. Table 15 gives both.

Two things are worth taking from this. The first is a caution for anyone reimplementing the method: a quadratic form whose reduction is unstated controls, through a saturating exponential, whether the adaptation runs, and the difference between the two readings here is nine orders of magnitude in the update rate. The second is that it does not change any result. With the reduction corrected the gate opens, and PeTTA still changes 10 predictions of 3412 with the normalisation channel disabled, and still loses 20.4 points with it enabled (Table 12). The method's behaviour on this data is decided by the normalisation channel, not by the gate.

We also note that the source covariance this statistic divides by is poorly conditioned here: 6.9% of its per-class entries fall below 10^{-6}, dead units in a 64-dimensional embedding of 13 features, and those entries carry 93.8% of the Mahalanobis mass. The summed reduction is what turns that conditioning problem into saturation, but a reader applying PeTTA to a low-dimensional sensor problem should check both.

DIGA: published hyperparameters do not transfer. Table 16 crosses the two readings of DIGA's prototype similarity with two hyperparameter settings.

The literal inner-product reading yields a negative MCC, confirming that $\langle \cdot, \cdot \rangle$ must denote a dissimilarity. More substantively, DIGA's published hyperparameters lose 8.8 macro-F_1 points against retuned values on this data. DIGA's own evaluation makes a point of using a single global parameter set across all experiments, in explicit contrast to competitors tuned per source–target pair. That property does not survive the move to one-dimensional sensor data, and recovering the loss required tuning on target labels, which deployment does not provide.

6.8. Where the papers and their released code disagree

Reproducing these three methods surfaced three places where a paper and its own released implementation specify different things. Each one changes a result here, so we record them rather than silently picking a side.

Table 16
DIGA under two readings of Eq. 8 and two hyperparameter settings. Source-only is macro-F_1 0.4641, MCC 0.4679.

Reading of $\langle\cdot,\cdot\rangle$	Hyperparameters	Changed	macro-F_1	MCC
Inner product (literal)	published 0.8/0.9	2573/3412	0.2151	0.1840
Inner product (literal)	retuned 0.95/0.50	3404/3412	0.0075	−0.1674
Squared distance	published 0.8/0.9	1324/3412	0.3932	0.3610
Squared distance	retuned 0.95/0.50	357/3412	**0.4808**	**0.4867**
Cosine	published 0.8/0.9	931/3412	0.3232	0.3399
Cosine	retuned 0.95/0.50	263/3412	0.4750	0.4825

PETAL's coefficient. Algorithm 2 places $\bar{\lambda}$ on the data term with the prior fixed at unity. The released code computes `loss_H + spw·para_loss` with `spw` $= 10^{-9}$, which is the same objective scaled, with the coefficient on the prior instead. The published configuration therefore corresponds to $\bar{\lambda} \approx 10^9$, not to any value in Table 14's range. The sweep in that table is still informative about what target-label selection does, but it is not a sweep through the authors' operating point.

PETAL's pseudo-label rule. The paper describes a per-sample confidence test. The released code applies a *batch-level* one, `anchor_prob.mean(0) < ap` with $ap = 0.92$, so the augmentation-averaged label replaces the entire batch or none of it. We follow the code. Under a per-sample threshold at 0.5, which is the natural reading of the prose, the augmentation branch fires on under a quarter of samples here and the $K = 32$ augmentations are close to inert.

PeTTA's reduction. Eq. 6 is a quadratic form, which reads as a sum over the embedding dimension. The released implementation averages. As Section 6.7 shows, on this data the two differ by nine orders of magnitude in the resulting update rate, because the statistic passes through a saturating exponential. We report the averaged version as the published method.

One general point is worth drawing out. All three discrepancies are in the direction of *suppressing* adaptation if a reimplementer follows the prose: a coefficient that reads as a regulariser weight but scales the likelihood, a confidence test that reads as per-sample but is per-batch, and a reduction that reads as a sum but is a mean. A negative result obtained from a paper alone, without its code, is therefore not safe to report as a property of the method. We obtained exactly such a result from each of these three before checking, and all three dissolved on contact with the released code.

6.9. How much of DIGA's gain survives implementation choices?

Two steps of DIGA are specified in a way that admits more than one implementation, and the difference is not cosmetic.

The first is the DAM. Eq. 2 blends batch-normalisation *statistics*, after which the input is normalised once. An alternative is to normalise twice, once with source statistics and once with batch statistics, and blend the two normalised *outputs*. These are different operations, and a reference implementation in circulation takes the second route while citing the first equation.

The second is the fusion step. Eq. 4 combines *probabilities*. The same implementation combines a cosine similarity, bounded in $[-1, 1]$, with unbounded classifier logits, so the relative weight of the two terms depends on logit scale rather than on λ_F alone.

Table 17 evaluates all four combinations under the protocol of Table 7.

The ordering is the one the equations predict: the reading that follows both published steps is best and the reading that follows neither is indistinguishable from doing nothing. But the spread across the four, 0.5 points, is the same size as the effect being measured, and no reading separates from zero on this protocol. We report DIGA throughout using the published equations, which is the reading most favourable to the method. The point is not that one reading is correct. It is that the adaptation gains at issue in this study are of a size comparable to the ambiguity in the method's own description, so a reader reimplementing DIGA from its prose may reasonably obtain no measurable benefit at all.

6.10. Does any hyperparameter setting change the conclusion?

The results so far compare each method at its published setting and, where the manuscript differs, at one alternative. That leaves open whether the activity-task numbers describe the methods or one point in their configuration space. Table 18 answers it by searching each method's own exposed hyperparameters: DIGA over $\lambda_{BN}\times\mathcal{P}_0\times\rho_P\times K$ (64 settings), PETAL over $\bar{\lambda} \times \pi \times \delta$ (24), and PeTTA over $\alpha_0 \times \lambda_0$ and learning rate (16). Each setting is run in both transfer directions with three seeds under the protocol of Table 7, and is scored two ways: by the best target macro-F_1 in the grid, which needs target labels and is therefore an upper bound and not a result, and by the configuration with the lowest mean prediction entropy on the target stream, which needs none and is what a deployment can reach.

Three things follow.

For PETAL and PeTTA the grids are flat at zero. No setting in PETAL's published space moves more than 36 of 3412 predictions, and none in PeTTA's moves more than 38; the largest target gain anywhere in either grid is 0.3 points, below the seed spread of the source model itself. The analysis in Section 6.7 attributes this to the update rules

Table 17
DIGA under four readings of its two under-specified steps, streaming protocol (13-d layout-invariant features, last GroupKFold fold, no smoothing), 3 seeds. Source-only is 0.4944. The four readings share identical source checkpoints, so each interval is on the paired difference and is narrower than the across-seed spread of any single column ($\sigma \approx 0.022$). No reading yields an interval excluding zero.

DAM	Fusion	macro-F_1	95% CI on Δ
blend statistics (Eq. 2)	probabilities (Eq. 4)	**0.4996**	[−0.0008, +0.0110]
blend outputs	probabilities	0.4969	[−0.0043, +0.0093]
blend statistics	similarity + logits	0.4961	[−0.0035, +0.0073]
blend outputs	similarity + logits	0.4946	[−0.0058, +0.0065]

Table 18
Full hyperparameter grids on the activity task, Δmacro-F_1 relative to source-only, averaged over three seeds. *help* counts settings with a positive mean. *oracle* is the grid maximum, which requires target labels; *entropy* is the setting chosen by lowest mean target prediction entropy, which does not. *changed* is the mean number of predictions altered out of 3412.

			Δmacro-F_1			
Method	Direction	help	median	oracle	entropy	changed
source-only macro-F_1 0.4944 ($S_1 \to S_2$), 0.3226 ($S_2 \to S_1$)						
DIGA	$S_1 \to S_2$	11/64	−0.0176	+0.0112	+0.0031	679
DIGA	$S_2 \to S_1$	64/64	+0.0224	+0.0349	+0.0197	578
PETAL	$S_1 \to S_2$	17/24	+0.0002	+0.0028	+0.0028	15
PETAL	$S_2 \to S_1$	10/24	−0.0001	+0.0009	−0.0009	20
PeTTA	$S_1 \to S_2$	9/16	+0.0000	+0.0003	+0.0000	7
PeTTA	$S_2 \to S_1$	4/16	+0.0000	+0.0002	−0.0003	5

and not to tuning, and the grids are consistent with that: a method whose data term is switched off by $\bar{\lambda} = 0$, or whose adaptation rate is driven to $\alpha_t \approx 0$ by a saturated gate, does not become active at a different π or λ_0. Notably, the oracle rule selects $\bar{\lambda} = 0$ for PETAL in the $S_1 \to S_2$ direction: the one value at which, by Eq. 5, the target stream enters no gradient.

For DIGA the grid is wide, and which way it points is decided by the source model rather than by the hyperparameters. Transferring from the stronger source, 53 of 64 settings lose and the median costs 1.8 points; transferring from the weaker one, all 64 gain and the median adds 2.2. Placing this beside the occupancy grid of Section 6.7 gives the same ordering across both tasks:

Source	Quality	Helped
occupancy S_1	MCC 0.6628	0/64
activity S_1	macro-F_1 0.4944	11/64
activity S_2	macro-F_1 0.3226	64/64

The fraction of a published grid that helps tracks how much headroom the direction has. Entering S2 the cross-building penalty is 0.6 points (Section 6.3) and 53 of 64 settings lose; entering S1 it is 21.3 points and all 64 gain. Read over whole configuration spaces rather than at single points, normalisation-statistic adaptation is not a property of a method that one tunes into working, it is a trade of source-fitted statistics for target-fitted ones, and whether that trade pays depends on which of the two is better.

Third, the two selection rules disagree, and the direction of the disagreement matters for how such results should be read. Label-free selection recovers 28% of DIGA's oracle gain in the forward direction and 56% in the reverse; for PETAL and PeTTA it returns a negative number in the reverse direction where the oracle returns a positive one. An entropy criterion prefers confident predictions, and a method can raise confidence while lowering accuracy, so the gap is expected, not anomalous. Its size is the point: the headroom a grid search reports is between a quarter and a half larger than what a system without target labels can select, which is the condition TTA is defined under.

6.11. Occupancy estimation

Table 19 runs the identical model and methods under three normalisation protocols.

Three results follow from Table 19, all of them about the forward direction; the reverse is taken up below. No method improves on the source model under any of the three, and every one of the four is below the baseline in every column. The choice of normalisation is worth +3.2 MCC points, but it is worth that to *every* method equally, including the unadapted baseline. It is normalisation, not adaptation, and a protocol that fits the scaler on target statistics will credit it to whichever method happens to be under test. The ordering also matches the activity task: PeTTA loses most (8.0 points), PETAL and AdaBN sit together, and DIGA loses least, which is the ordering Section 6.7 predicts from how much of the source normalisation each keeps.

Running the transfer in the reverse direction, training on the target office and evaluating on the source office, changes

Table 19
Occupancy estimation, MCC on the target test set ($n = 150$, 34 minority), under three normalisation protocols. Always predicting "not occupied" gives MCC 0.000.

Method	Fitted on source	Fitted on target train	Fitted on target train + test
Source-only	**0.6628** ± 0.013	**0.6892** ± 0.014	**0.6951** ± 0.020
AdaBN	0.6412	0.6461	0.6369
DIGA	0.6429	0.6616	0.6694
PETAL	0.6382	0.6233	0.6428
PeTTA	0.5832	0.5829	0.6182

Table 20
Occupancy in both transfer directions, MCC, source-fitted normalisation, 5 seeds. The only thing that differs between the two columns is which office supplies the source model. Always predicting "not occupied" gives MCC 0.000 in both.

Method	source → target	target → source
Source-only	**0.6628**	−0.0763
AdaBN	0.6412	0.3302
DIGA	0.6429	0.2019
PETAL	0.6382	**0.3509**
PeTTA	0.5832	0.2353

the picture completely, and in a way that is the most direct evidence in this paper for its central claim.

In the reverse direction the source checkpoint is not merely weak but worse than chance: trained on the target office, it reaches MCC −0.0763 on the source office, meaning its predictions are anti-correlated with the labels. Against that starting point every method gains: PETAL by 42.7 MCC points, AdaBN by 40.7, PeTTA by 31.2 and DIGA by 27.8. In the forward direction, from a checkpoint at 0.6628, the same four lose 2.0 to 8.0.

The contrast is as clean as this dataset allows: identical methods, identical protocol, identical features, two offices, and the only variable is whether the source model was any good. When it is, adaptation has nothing to contribute and the normalisation-based methods actively harm it; when it is broken, the same methods recover a large part of the deficit. This mirrors the activity task, where the same methods cost 9 to 20 macro-F_1 points against a sound checkpoint, and it supports reading adaptation gain as a measure of source-model deficiency and not of adaptation quality. A method evaluated only against a weak source model will appear to work.

Taken together the four conditions in this study order by the quality of the checkpoint being adapted, and not by task, direction or method:

Condition	Source quality	AdaBN	DIGA	PETAL	PeTTA
Occupancy, forward	MCC 0.663	−2.2	−2.0	−2.5	−5.0
Activity, $S_1 \rightarrow S_2$	macro-F_1 0.494	−18.0	−8.6	−18.3	−20.4
Activity, $S_2 \rightarrow S_1$	macro-F_1 0.323	−10.9	+2.1	−12.0	−10.7
Occupancy, reverse	MCC −0.076	+40.7	+27.8	+42.7	+31.2

Sixteen measurements across two tasks, two directions and four methods. Every method helps against the broken checkpoint and every method hurts against the sound ones; the only positive entry anywhere else is DIGA in the direction with 21 points of headroom.

7. Discussion

The headline result of this study is not about any one method. It is that the quantity these methods are asked to repair, the cross-building performance drop, is routinely measured in a way that substantially overstates it, and that most of what remains once it is measured consistently is attributable to feature design and not to anything TTA can reach.

Measured against the right ceiling, the cross-building penalty is 0.6 points entering one apartment and 21.3 entering the other (Section 6.3), against the 83 points implied by comparing a leakage-inflated source figure with an honest target one. In the forward direction a model built from thirteen layout-invariant statistics performs on the second apartment as well as a model trained on that apartment's own days. The in-domain ceiling then becomes the binding constraint: one cannot improve target performance much beyond it, which is why our attempts to push the source model further (richer invariant features, class weighting, wider networks) gained little. Only causal smoothing helped, and modestly.

The asymmetry is what we would most want a reader to take away, because it decides what adaptation can do. Where the penalty is 0.6 points there is no headroom, and across DIGA's full published grid 53 of 64 settings lose ground. Where it is 21.3 points all 64 gain. Occupancy estimation reproduces the same split on a second task and a different model family. A reported adaptation gain is therefore first of all a statement about the gap the method was given, and only afterwards about the method.

This also reframes what per-sensor features cost, and what in-domain validation is worth. In-domain those features are valuable: the best source-validation score in Table 4 belongs to a set that retains them. Across buildings they are worse than useless, scoring below chance alone in both directions. The consequence for model selection is real but narrower than the extremes of those tables suggest. Pooled over all 70 configurations in this study, source validation and target performance are positively correlated ($\rho = +0.63$, $p < 0.001$); among competitive configurations the relationship weakens to $\rho = +0.27$ ($p = 0.11$), and the best-validating configuration is not the best-transferring one in either direction. In-domain validation reliably separates working configurations from broken ones. Between reasonable alternatives, which is the choice a practitioner actually faces, it carries little information about which will survive a change of building, and a held-out building is what settles it.

On the methods themselves, the three failures are instructive precisely because they are different from one another. PETAL's is a reading failure with a sharp edge: a coefficient that looks like a regulariser weight scales the likelihood, and the hyperparameter search most practitioners would run, selecting on target performance, actively prefers the value that disables adaptation. PeTTA's is dimensional: a divergence statistic that behaves sensibly when the domain gap is a corruption of the same image saturates when the gap is a different building, and because the mechanism fails silently the loss curves look unremarkable throughout. DIGA's is about generality: it works, but the single global parameter set it advertises does not, and the gap between its published and retuned settings is larger than the gain adaptation produces.

A practical consequence runs through all three. Hyperparameter selection on held-out target labels, which is standard in this literature, is not merely optimistic. It can select degenerate configurations, as the PETAL sweep shows, and it can make a method appear to transfer when what transferred was the tuning. A label-free criterion should be reported alongside any oracle-tuned number.

The occupancy results in both directions (Table 20) are the cleanest demonstration of this available here, because they hold everything constant but the quality of the source model. Starting from a checkpoint at MCC 0.658, adaptation costs roughly two points; starting from one at -0.078, the same methods gain twenty-seven to forty-one. The methods did not change. What changed is how much room there was.

What generalises, and what does not. The findings above differ in how far they travel, and it is worth separating them.

Two do not depend on this dataset at all. PETAL's $\bar{\lambda}$ multiplies the cross-entropy in its published objective (Eq. 5), so $\bar{\lambda} = 0$ removes the data term on any dataset, in any modality; The dataset-specific part is narrower than it may appear: selection on target labels prefers $\bar{\lambda} = 0$ whenever the source model already outperforms what self-training produces, which is a condition that can be checked, not a coincidence of this data. PeTTA's gating function likewise saturates whenever the summed Mahalanobis statistic is large, which is a property of summing over embedding dimensions rather than of these apartments, though whether the statistic reaches that regime does depend on how far apart the domains are, and the authors' own traces on image corruptions sit at 0.8–1.0 rather than at unity.

The evaluation result is a property of a procedure, not of a dataset: stride-1 windowing with a random validation split places near-duplicates across the split whenever windows overlap, which is common in sensor-based HAR. That overlapping windows bias cross-validation in activity recognition is established [1]; what we add is the magnitude on this pipeline, 46 points of source macro-F_1, and the separation of the split from the window definition, which contributes under two.

The asymmetry of the cross-building penalty is different in kind. It is measured here on two apartments, so the specific values of 0.6 and 21.3 points belong to this pair and nothing follows about their magnitude elsewhere. What we would expect to generalise is the procedure and its consequence: a cross-building penalty should be measured against a model trained on the target building at matched sample size, not against source validation, and until it is measured that way an adaptation gain cannot be separated from the headroom it was handed. Establishing how often the penalty is negligible requires datasets with more than two environments.

Limitations. The study rests on one HAR dataset and two apartments from it. Running both transfer directions rules out a direction-specific fluke but says nothing about other buildings, other sensor modalities, or richer activity sets; external validity beyond these two apartments is untested, and the dataset contains no further homes. The occupancy data is private, so that section is not independently reproducible from the released code. Our best configuration improves on the simpler one by 3.3 macro-F_1 points against a between-fold standard deviation of 0.067, and we present it as a modest gain and not a result. And the `cleaning` class, with 113 target samples, is not learned by any method we tried; whether that is a sample-size limit or a representational one, we cannot say from these data.

8. Conclusion

We evaluated three image-domain test-time adaptation methods on cross-building human activity recognition and occupancy estimation, and found that the evaluation protocol dominated the comparison. Stride-one windowing with a random validation split inflates source-domain macro-F_1 from roughly 0.53 to 0.998; grouping the split by recording day removes the effect, and the window definition is nearly irrelevant to it. Making features causal, which the online premise of TTA requires, improves target transfer by 7.3 points rather than costing anything.

Measured against a model trained on the target building at matched sample size, the cross-building penalty is 0.6 points entering one apartment and 21.3 entering the other.

One of our two directions therefore contains essentially no domain shift, and what the conventional source-minus-target figure counts as an 8.8-point gap there is mostly the difference between the two apartments as prediction problems. Most of the apparent gap that does exist is attributable to features encoding sensor identity, which score below stratified-random guessing on the target in both directions.

The methods follow that asymmetry, and they do it through one shared mechanism. Each of the four adapts batch-normalisation statistics, and each released implementation makes that its primary channel rather than an incidental detail. Disabled, with everything else unchanged, PETAL and PeTTA alter fewer than 25 of 3412 predictions in either direction. Enabled as their own code configures it, PETAL falls within 0.3 macro-F_1 points of plain AdaBN and PeTTA within 1.1, having changed much the same predictions. The cost rises monotonically with the share of target statistics mixed in, from -5.7 points at pure source statistics to -33.0 at pure target ones. DIGA is the exception only because it keeps most of the source estimate, and its gain comes from its prototype classifier rather than its normalisation. Occupancy estimation reproduces all of this on a second task, and supplies the clearest version of the pattern: against a checkpoint worse than chance every method gains between 28 and 43 MCC points, and against every sound checkpoint every method loses. A further apparent gain proves attributable to a target-fitted normalisation worth 3.2 MCC points to every method, the unadapted one included.

We do not conclude that test-time adaptation is unsuitable for smart buildings. We conclude that it was being asked to repair a problem largely created by evaluation and feature design, and that fixing those leaves little for adaptation to contribute. Future work should look at making building-specific information transferable instead of discarding it, aligning sensors across sites by functional role using unlabelled co-activation structure, since the points separating our layout-invariant features from the best in-domain configuration are exactly the information currently being thrown away.

Reproducibility

Code, the raw sensor data and every result file are available at `https://github.com/mariemkbaier/TTA-Test-Time-Adaptation`. The repository reproduces the full analysis from the raw CSVs in one pass, each table above names the script and result file that generated it. The occupancy dataset is private: it was collected at the Grenoble Institute of Technology and is not ours to redistribute, so that section alone is reported but not independently reproducible. The three adaptation methods are implemented in `src/tta.py` (as commonly applied) and `src/tta_faithful.py` (at the hyperparameters and equations each paper publishes); the corrected PETAL of Section 6.7 is `src/petal_fixed.py`.

Acknowledgements

This research was supported by the Natural Sciences and Engineering Research Council of Canada (NSERC).

A. A larger layout-invariant feature set

We also evaluated a 60-dimensional invariant set adding a sorted activation profile (the per-sensor window means sorted in descending order, which is invariant to any permutation of sensor indices), run-length and recency structure, and the window statistics recomputed at $W \in \{5, 10, 40\}$. It does not help:

Permutation invariance in sensor index is evidently not sufficient: S1 has 70 sensors and S2 has 75, placed in differently shaped rooms, so "how many sensors are busy and how concentrated the activity is" is itself building-specific. Inverse-frequency class weighting also failed, reducing macro-F_1 to 0.3852 and *lowering* balanced accuracy from 0.4818 to 0.4280, most plausibly because it pushes the model toward `cleaning`, which is not learnable from 113 target samples, at the cost of the four classes that do work.

References

[1] N. Y. Hammerla, T. Plötz, Let's (not) stick together: pairwise similarity biases cross-validation in activity recognition, in: Proceedings of the 2015 ACM International Joint Conference on Pervasive and Ubiquitous Computing, 2015, pp. 1041–1051.

[2] M. Manfren, M. Sibilla, L. Tronchin, Energy modelling and analytics in the built environment—a review of their role for energy transitions in the construction sector, Energies 14 (3) (2021) 679.

[3] B. Amangeldy, T. Imankulov, N. Tasmurzayev, G. Dikhanbayeva, Y. Nurakhov, A review of artificial intelligence and deep learning approaches for resource management in smart buildings, Buildings 15 (15) (2025) 2631.

[4] J. Dridi, M. Amayri, N. Bouguila, Transfer learning for estimating occupancy and recognizing activities in smart buildings, Building and Environment 217 (2022) 109057.

[5] M. Amayri, S. Ploix, N. Bouguila, F. Wurtz, Estimating occupancy using interactive learning with a sensor environment: real-time experiments, IEEE Access 7 (2019) 53932–53944.

[6] J. Dridi, M. Amayri, N. Bouguila, Unsupervised clustering-based domain adaptation for estimating occupancy and recognizing activities in smart buildings, Journal of Building Engineering 85 (2024) 108741.

[7] R. Shahbazian, I. Trubitsyna, Human sensing by using radio frequency signals: a survey on occupancy and activity detection, IEEE Access 11 (2023) 40878–40904.

[8] T. Hong, D. Yan, S. D'Oca, C. Chen, Ten questions concerning occupant behavior in buildings: the big picture, Building and Environment 114 (2017) 518–530.

[9] J. Liang, R. He, T. Tan, A comprehensive survey on test-time adaptation under distribution shifts, International Journal of Computer Vision 133 (1) (2025) 31–64.

[10] W. Wang, Z. Zhong, W. Wang, X. Chen, C. Ling, B. Wang, N. Sebe, Dynamically instance-guided adaptation: a backward-free approach for test-time domain adaptive semantic segmentation, in: CVPR, 2023, pp. 24090–24099.

[11] D. Brahma, P. Rai, A probabilistic framework for lifelong test-time adaptation, in: CVPR, 2023, pp. 3582–3591.

[12] T.-H. Hoang, D. M. Vo, M. N. Do, Persistent test-time adaptation in recurring testing scenarios, in: NeurIPS, Vol. 37, 2024.

[13] D. Wang, E. Shelhamer, S. Liu, B. Olshausen, T. Darrell, Tent: fully test-time adaptation by entropy minimization, in: ICLR, 2021.

[14] Q. Wang, O. Fink, L. Van Gool, D. Dai, Continual test-time domain adaptation, in: CVPR, 2022, pp. 7201–7211.

[15] J. Liang, D. Hu, J. Feng, Do we really need to access the source data? Source hypothesis transfer for unsupervised domain adaptation, in: ICML, Vol. 119 of PMLR, 2020, pp. 6028–6039.

Table 21
Richer layout-invariant features, S1→S2, GroupKFold, 3 seeds.

Feature set	d	Target macro-F_1
Global + temporal	13	$\mathbf{0.4350} \pm 0.061$
+ sorted activation profile	28	0.4245 ± 0.039
+ run-length structure	33	0.4227 ± 0.027
+ multi-scale windows	60	0.4188 ± 0.028

[16] O. Abderrahim, J. Dridi, M. Amayri, N. Bouguila, Occupancy estimation and activity recognition in smart buildings using open set domain adaptation, Journal of Building Engineering 112 (2025) 113784.

[17] O. Abderrahim, J. Dridi, M. Amayri, N. Bouguila, Scalable activity recognition in smart buildings via generalized domain adaptation of IoT sensor data, Energy and Buildings 351 (2026) 116692.

[18] O. Abderrahim, J. Dridi, M. Amayri, N. Bouguila, Open set domain adaptation and universal domain adaptation for enhancing activity recognition in smart buildings, in: Proceedings of the 19th IBPSA Conference, Brisbane, Australia, 2025. https://doi.org/10.26868/25222708.2025.1331.

[19] S. Wang, J. Wang, H. Xi, B. Zhang, L. Zhang, H. Wei, Optimization-free test-time adaptation for cross-person activity recognition, Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies 7 (4) (2023) 183.

[20] E. M. Tapia, S. S. Intille, K. Larson, Activity recognition in the home using simple and ubiquitous sensors, in: Pervasive Computing, Springer, Berlin, Heidelberg, 2004, pp. 158–175.

[21] D. Chicco, G. Jurman, The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation, BMC Genomics 21 (1) (2020) 6.